

Received 11 September 2025, accepted 13 October 2025, date of publication 22 October 2025,
date of current version 19 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3624130

RESEARCH ARTICLE

Adaptive Dynamic Window Approach for Mobile Robot Motion Control Using Particle Swarm Optimization

SAMER S. SARKIS¹, OMAR S. ELFAYOUMI¹, MOHAMMAD A. JARADAT²,
AND BARA J. EMRAN³, (Member, IEEE)

¹Mechatronics Engineering Graduate Program, American University of Sharjah, Sharjah, United Arab Emirates

²Department of Mechanical Engineering, American University of Sharjah, Sharjah, United Arab Emirates

³Department of Electrical Engineering, American University of Sharjah, Sharjah, United Arab Emirates

Corresponding author: Mohammad A. Jaradat (mjaradat@aus.edu)

This work was supported in part by American University of Sharjah (AUS), Faculty Research Grant (FRG) Program under Grant FRG24-E-E91; and in part by the Open Access Program from American University of Sharjah.

ABSTRACT Autonomous navigation for mobile robots in dynamic and unknown environments requires a robust and adaptable path planning approach able to handle real world various applications. Among the most widely deployed local path planning approaches is the Dynamic Window Approach (DWA), known for its effectiveness in reactive obstacle avoidance and local trajectory planning and generation. DWA approach obtains its optimal collision free path by optimizing an objective function, which manages the trade-off between reaching the goal and obstacle avoidance. This optimization process depends on the appropriate preselection of a set of fixed weights within the DWA objective function; this set of weights is normally preselected for static or controlled environment situations. However, this fixed preselection process of the weights limits the adaptability performance of the DWA in complex or dynamic environment. To address this limitation, this paper proposes an intelligent adaptive DWA framework for mobile robot navigation in both static and dynamic environments (PSO-DWA). The proposed framework incorporate an online Particle Swarm Optimization (PSO) approach that dynamically adjusts the DWA objective function weights based on environmental feedback to improve its adaptability to environmental changes and efficiency of the robot performance. The performance of the proposed intelligent adaptive PSO-DWA was compared against the classical DWA and another adaptive variant DWA approach based on genetic algorithm (GA-DWA). The demonstrated simulation results show that there was a substantial reduction in robot travel time by 30% and an increase in the navigation success rate by 20%, with a 10% increase in average robot velocity over the classical DWA, and consistently outperformed GA-DWA especially in dense and dynamic environments. Theses finding were validated through real-time experiments conducted through various static and dynamic environment testing scenarios using a mobile robot platform, confirming the effectiveness and adaptability of the proposed framework.

INDEX TERMS Collision avoidance, intelligent adaptive dynamic window approach, particle swarm optimization, mobile robots.

I. INTRODUCTION

Trajectory planning refers to the process in which the robot tries to determine a viable path from an initial state to a

goal state while avoiding obstacles. These obstacles can either be static, which makes them relatively easier to handle in the planning process as they never change their state, or dynamic, which is more challenging to predict their state changes over time. One way to overcome this limitation is to keep re-planning whenever the state of a dynamic obstacle

The associate editor coordinating the review of this manuscript and approving it for publication was Xu Jing¹.

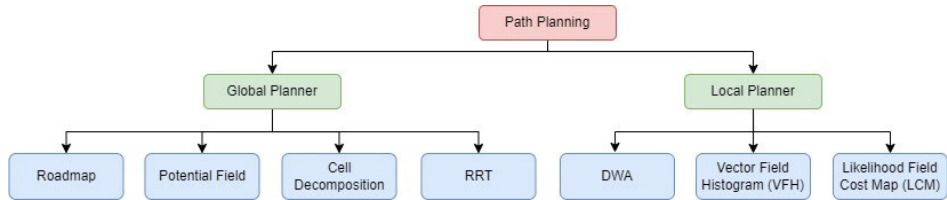


FIGURE 1. Commonly used path planning algorithms.

TABLE 1. Comparison of various DWA path planning frameworks.

Integration Strategy	Advantages	Limitations	Adaptive Weights	Approach	Validation Approach	Reference
Global A* planning combined with local DWA execution	Improves path optimality by leveraging A* and DWA, better trajectory initialization	Limited reactivity to unexpected dynamic changes, depends on A* path validity	No	-	Simulation	[21]
Reinforcement learning (Q-learning) to update motion parameters	Learns optimal behaviors in congested scenarios; adapts over time	Requires extensive training episodes, increased computational complexity	Yes	Online	Simulation + Experimental	[15]
Environment-responsive DWA with self-adjusting parameters	Automatically adjusts parameters based on sensor feedback; reduces manual tuning	Reactive but not predictive, parameter changes may lag in real-time	Yes	Online	Simulation + Experimental	[18]
Adaptive parameter tuning of DWA via fuzzy inference system	Provides real-time adaptation to changing environments through linguistic rules	Performance hinges on fuzzy rule quality, lacks learning capability	Yes	Online	Simulation	[19]
Formation-aware DWA with dynamic obstacle handling	Maintains group cohesion in multi-robot formations; robust to dynamic obstacles	Tailored for formation control, less applicable to single-robot setups	No	-	Simulation + Experimental	[20]
Hybridization of improved PSO with DWA cost weights	Globally optimizes trajectory smoothness and safety using PSO	Sensitive to PSO settings, high computational demand during tuning	No	Offline	Simulation	[25]
PSO-based hybrid optimization of DWA parameters using linear and quadratic fitness functions	improves adaptability, balances speed, success rates, and obstacle clearance across diverse environments	Adds computational overhead	Yes	Online/Offline	Simulation + Experimental	Proposed in this paper

changes. However, this approach suffers from significant high computational cost and increases the system latency. Some researchers have attempted to address similar challenges in other planning contexts. For example, using metaheuristic optimization in combination with Dubins paths to generate safe emergency landing trajectories under dynamic constraints [1]. Similarly, in [2], unmanned aerial vehicles (UAVs) can adapt their flight paths in windy environments to maximize endurance, showing how trajectory planning optimization can benefit from environmental adaptation. The obstacle avoidance challenge has been tackled through several local path planning approaches as shown in figure 1. For example, Vector Field Histogram (VFH) [3], Likelihood Field Cost Map (LCM) [4], and DWA [5], which is widely recognized for its ability to balance dynamic constraints such as acceleration and velocity limits with collision avoidance in unstructured environments.

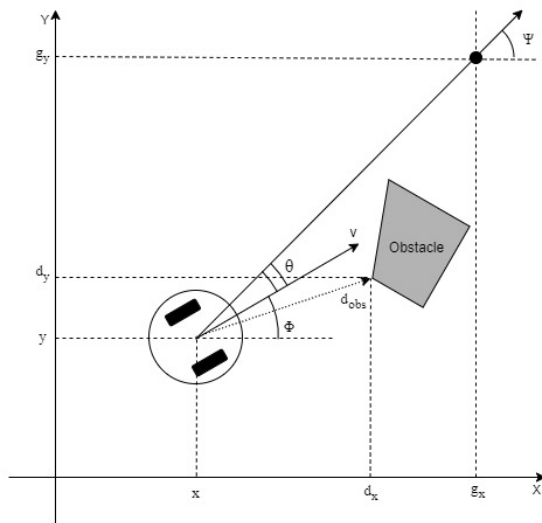
The classical DWA focuses on local trajectory optimization, where a set of trajectories and feasible velocity commands is computed using a dynamic window [5].

Specifically, the goal is to compute a set of control inputs to the robot (V, ω) , where V is the linear velocity and ω is the angular velocity. These velocities must be within a feasible window determined by the robot’s maximum linear and angular acceleration and deceleration capabilities over a short time step. Given these velocity limits, the next step is to generate a set of trajectories based on the robot’s kinematic constraints.

The robot’s motion is determined by evaluating trajectories that take into account the current velocity, acceleration limits, and the configuration of nearby obstacles. The trajectory that maximizes the robot’s progression toward the goal, while avoiding collisions and respecting dynamic constraints, is then selected for execution. Essentially, the challenge becomes selecting the optimal trajectory based on the maximization of a total cost computed by a cost function. This cost function considers alignment of the trajectory with the target direction, the minimum distance to obstacles to ensure collision-free paths, and optimization of velocity [5].

TABLE 2. Comparison of various optimization algorithms used in trajectory or parameter optimization.

Algorithm	Nature	Advantages	Limitations	Reference
Particle Swarm Optimization (PSO)	Swarm-based, stochastic	Fast convergence, simple to implement, gradient-free, less prone to local optima, robust with adaptive inertia weights, and balances global exploration with local exploitation	Scalability in high dimensions, Stochastic variability	[25]
Genetic Algorithm (GA)	Evolutionary, stochastic	Effective in global search, adaptable to various problem spaces	Requires encoding/decoding, slower convergence	[30]
Simulated Annealing (SA)	Probabilistic, stochastic	Capable of escaping local minima, simple setup	Performance highly dependent on cooling schedule, slower	[31]
Bayesian Optimization	Model-based, probabilistic	Sample-efficient, suitable for expensive black-box functions	Computationally intensive in high-dimensional spaces	[32]
Covariance Matrix Adaptation Evolution Strategy (CMA-ES)	Evolutionary, stochastic	Well-suited for noisy, high-dimensional problems, no derivatives needed	High computational cost per iteration	[33]

**FIGURE 2.** Geometric representation of robot orientation, goal direction, and obstacle distance in a 2D environment.

Although classical DWA is easy to implement, it suffers from several limitations. First, it is susceptible to getting stuck in local minima, particularly in narrow passages. Without a global perspective, the robot might not reach the goal [6]. Second, the assumption of static obstacles limits the performance of the classical DWA [7]. In environments where dynamic obstacles move unpredictably, it becomes necessary to perform the re-planning process at a very high frequency, thereby increasing computational complexity and reducing real-time performance. In addition, the original formulation of DWA is better suited for holonomic robots. For non-holonomic robots (e.g., differential-drive robots), it struggles with orientation constraints and can result in inefficient paths [8].

Thus, to overcome these limitations, versatile approaches have been developed. These approaches incorporate additional layers of intelligence, such as learning-based methods

Algorithm 1 Dynamic Window Approach (DWA)

Require: Current robot state $\xi(t)$, goal position g , robot model parameters, DWA parameters (α, β, γ)

Ensure: Optimal control (v^*, ω^*)

```

1: while goal not reached do
2:    $u \leftarrow \emptyset$ ;  $max\_score \leftarrow -\infty$ 
3:   for  $v \in Arrange(v_{min}, v_{max}, v_{reso})$  do
4:     for  $\omega \in Arrange(\omega_{min}, \omega_{max}, \omega_{reso})$  do
5:        $traj \leftarrow CalculateTraj(x, [v, \omega])$ 
6:        $G \leftarrow GoalBenefit(g, traj)$  {Eq. (2) - (3) - (4)}
7:        $O \leftarrow ObstacleBenefit(traj)$  {Eq. (5)}
8:        $V \leftarrow VelocityBenefit(traj)$ 
9:        $score \leftarrow \alpha G + \beta O + \gamma V$  {Eq. (1)}
10:      if  $score > max\_score$  then
11:         $max\_score \leftarrow score$ 
12:         $u \leftarrow [v, \omega]$ 
13:         $traj^* \leftarrow traj$ 
14:      end if
15:    end for
16:  end for
17:   $ApplyControl(u, \Delta t)$ 
18:   $UpdateState(\xi(t))$ 
19: end while
20: return  $u, traj^*$ 

```

and the fusion of other planning algorithms. The aim of these adaptive approaches is to make DWA more responsive to dynamic and uncertain environments. They also improve computational efficiency and provide more reliable and robust navigation performance [9].

In [10], Mai et al. improved the efficiency of the classical DWA by incorporating a density evaluation factor into the trajectory evaluation function. First, the use of multi-sensor technology and new evaluation algorithms capable of density perception. Then, the evaluation factor of the density change is fed into the path evaluation function. Moreover, to solve

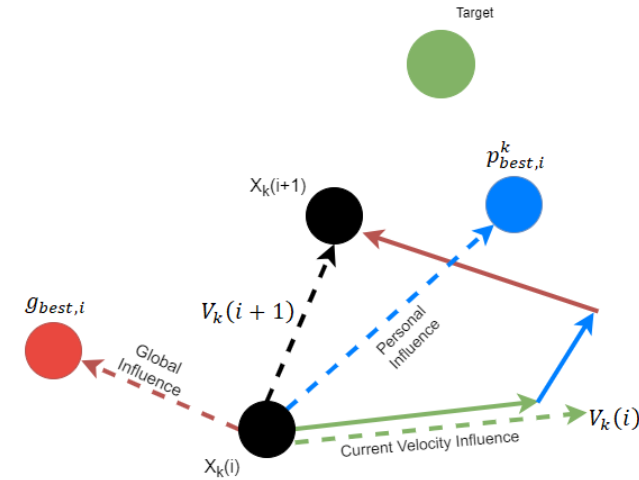


FIGURE 3. Velocity update process in PSO, illustrating the contributions of global best, personal best, and current velocity influences to the particle's next position.

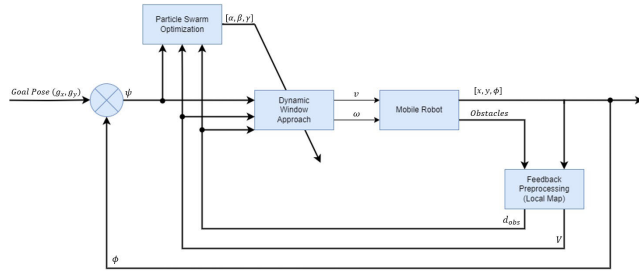


FIGURE 4. PSO-DWA-ON algorithm architecture.

highly unpredictable dynamic environments, a predictive model was introduced that predicts the trajectories of dynamic obstacles based on their current velocity and acceleration. In this interpretation, the dynamic window is recalculated on the basis of the predicted obstacle movements [11]. Another method proposed (VADWA) [12], to solve what is described as the detouring problem in AUVs. This refers to the situation where the robot fails to follow an optimal trajectory, instead taking a longer, inefficient route or getting stuck in a loop. Adaptive parameters and vector field histogram were used to solve this problem. In addition, a dynamic model was taken into consideration, which includes the input characteristics of the actuators.

The classical DWA implies a constant sampling time, which is a trajectory exploration limitation for Automated Guided Vehicles (AGVs). This limitation was solved by using the twin delayed deep deterministic policy gradient (TD3) to generate an additional trajectory. As a result, sampling utilization efficiency and planning effectiveness increased significantly [13]. One of these implemented improvements is the use of reinforcement learning and real-time sensor input to continuously change the weights of the cost function to prioritize different objectives. Hence, allowing fine-tuning of the parameters during execution,

Algorithm 2 PSO–DWA–OFF

Require: Map M , goal g ; bounds on (α, β, γ) ; PSO params: swarm size N , max iterations $I_{\max}$, inertia ω_p , gains c_1, c_2
Ensure: Fixed weights $(\alpha^*, \beta^*, \gamma^*)$ and runtime control (v, ω)

- 1: **Initialization:**
- 2: **for** $k = 1$ to N **do**
- 3: $X_k(0) \leftarrow \text{rand}[\alpha, \beta, \gamma]$ {random weights within bounds}
- 4: $V_k(0) \leftarrow 0$
- 5: $p_{\text{best},0}^k \leftarrow X_k(0)$
- 6: $J_k(0) \leftarrow \text{FitnessOffline}(X_k(0); M, g)$
- 7: **end for**
- 8: $g_{\text{best},0} \leftarrow \arg \min_k J_k(0)$
- 9: **for** $i = 0$ to $I_{\max} - 1$ **do**
- 10: **for** $k = 1$ to N **do**
- 11: $V_k(i+1) \leftarrow \text{Velocity update (Eq. (15))}$
- 12: $X_k(i+1) \leftarrow \text{Position update (Eq. (16))}$
- 13: $X_k(i+1) \leftarrow \text{ClampToBounds}(X_k(i+1))$
- 14: $J_k(i+1) \leftarrow \text{FitnessOffline}(X_k(i+1); M, g)$
- 15: **if** $J_k(i+1) < \text{FitnessOffline}(p_{\text{best},i}^k; M, g)$ **then**
- 16: $p_{\text{best},i+1}^k \leftarrow X_k(i+1)$
- 17: **else**
- 18: $p_{\text{best},i+1}^k \leftarrow p_{\text{best},i}^k$
- 19: **end if**
- 20: **end for**
- 21: $g_{\text{best},i+1} \leftarrow \arg \min_k \text{FitnessOffline}(p_{\text{best},i+1}^k; M, g)$
- 22: **end for**
- 23: $(\alpha^*, \beta^*, \gamma^*) \leftarrow g_{\text{best},I_{\max}}$
- 24: **Runtime DWA (maximize Score, Eq. (1)):**
- 25: **while** goal not reached **do**
- 26: $u \leftarrow \emptyset$; $\text{max_score} \leftarrow -\infty$
- 27: **for** $v \in \text{Arrange}(v_{\min}, v_{\max}, v_{\text{reso}})$ **do**
- 28: **for** $\omega \in \text{Arrange}(\omega_{\min}, \omega_{\max}, \omega_{\text{reso}})$ **do**
- 29: $\text{traj} \leftarrow \text{CalculateTraj}(x, [v, \omega])$
- 30: $G \leftarrow \text{GoalBenefit}(g, \text{traj})$ {Eq. (2) - (3) - (4)}
- 31: $O \leftarrow \text{ObstacleBenefit}(\text{traj})$ {Eq. (5)}
- 32: $V \leftarrow \text{VelocityBenefit}(\text{traj})$
- 33: $\text{score} \leftarrow \alpha^* G + \beta^* O + \gamma^* V$
- 34: **if** $\text{score} > \text{max_score}$ **then**
- 35: $\text{max_score} \leftarrow \text{score}$
- 36: $u \leftarrow [v, \omega]$
- 37: $\text{traj}^* \leftarrow \text{traj}$
- 38: **end if**
- 39: **end for**
- 40: **end for**
- 41: $\text{ApplyControl}(u, \Delta t)$
- 42: $\text{UpdateState}(\xi(t))$
- 43: **end while**
- 44: **return** u, traj^*

depending on the environment [14]. Another adaptation is to use reinforcement learning to change the weights of the cost function. Specifically, the implementation of Q-Learning and pre-learned Q-table to dynamically select optimal paths and weight coefficients to counteract changes in environmental conditions such as congestion levels, road width, and obstacle density [15]. This Q-learning approach has a very high computational cost and requires a long time for training. Alternatively, a deep convolution neural network can be trained with reinforcement learning algorithms to tune weights by treating the limitation as a data-driven problem, as shown in the Adaptive Dynamic Window Approach (ADWA) [16]. Field Dynamic Window Approach (F-DWA)

Algorithm 3 PSO–DWA–ON

Require: Goal g , robot model, control period Δt ; bounds for (α, β, γ) ; PSO params: swarm size N , inner iters K , inertia ω_p , gains c_1, c_2

Ensure: Control (v, ω) each cycle with online-updated $(\alpha_t, \beta_t, \gamma_t)$

- 1: **Initialization:** choose nominal weights $X_{\text{nom}} = [\alpha, \beta, \gamma]$
- 2: **for** $k = 1$ to N **do**
- 3: $X_k(0) \leftarrow$ sample near X_{nom}
- 4: $V_k(0) \leftarrow 0$
- 5: $p_{\text{best},0}^k \leftarrow X_k(0)$
- 6: **end for**
- 7: $g_{\text{best},0} \leftarrow \arg \min_k \text{FitnessOnline}(X_k(0))$
- 8: **while** goal not reached **do**
- 9: $\mathcal{M}_t \leftarrow \text{BuildLocalMap}()$ {obstacles, d_{obs} , velocities}
- 10: **Online PSO refinement** (K iterations):
- 11: **for** $i = 0$ to $K - 1$ **do**
- 12: **for** $k = 1$ to N **do**
- 13: $V_k(i+1) \leftarrow$ Velocity update (Eq. (15))
- 14: $X_k(i+1) \leftarrow$ Position update (Eq. (16))
- 15: $X_k(i+1) \leftarrow \text{ClampToBounds}(X_k(i+1))$
- 16: $J_k(i+1) \leftarrow \text{FitnessOnline}(X_k(i+1); \mathcal{M}_t, g)$
- 17: **if** $J_k(i+1) < \text{FitnessOnline}(p_{\text{best},i}^k; \mathcal{M}_t, g)$ **then**
- 18: $p_{\text{best},i+1}^k \leftarrow X_k(i+1)$
- 19: **else**
- 20: $p_{\text{best},i+1}^k \leftarrow p_{\text{best},i}^k$
- 21: **end if**
- 22: **end for**
- 23: $g_{\text{best},i+1} \leftarrow \arg \min_k \text{FitnessOnline}(p_{\text{best},i+1}^k; \mathcal{M}_t, g)$
- 24: **end for**
- 25: $[\alpha_t, \beta_t, \gamma_t] \leftarrow g_{\text{best},K}$
- 26: **DWA selection (maximize Score, Eq. (1)):**
- 27: $u \leftarrow \emptyset$; $\max_score \leftarrow -\infty$
- 28: **for** $v \in \text{Arrange}(v_{\min}, v_{\max}, v_{\text{reso}})$ **do**
- 29: **for** $\omega \in \text{Arrange}(\omega_{\min}, \omega_{\max}, \omega_{\text{reso}})$ **do**
- 30: $\text{traj} \leftarrow \text{CalculateTraj}(x, [v, \omega])$
- 31: $G \leftarrow \text{GoalBenefit}(g, \text{traj})$ {Eq. (2)–(4)}
- 32: $O \leftarrow \text{ObstacleBenefit}(\text{traj})$ {Eq. (5)}
- 33: $V \leftarrow \text{VelocityBenefit}(\text{traj})$
- 34: $\text{score} \leftarrow \alpha_t G + \beta_t O + \gamma_t V$
- 35: **if** $\text{score} > \max_score$ **then**
- 36: $\max_score \leftarrow \text{score}$
- 37: $u \leftarrow [v, \omega]$
- 38: $\text{traj}^* \leftarrow \text{traj}$
- 39: **end if**
- 40: **end for**
- 41: **end for**
- 42: ApplyControl($u, \Delta t$)
- 43: UpdateState($\xi(t)$)
- 44: **end while**
- 45: **return** u, traj^*

was developed with an adjusted objective function that also took into consideration the obstacles near the trajectory. This is done using histogram grid representation of obstacles to estimate the probability of collision of the trajectory [17]. Similarly, fuzzy logic can perform parameter tuning using the target distance and feasible velocity space and a set of fuzzy rules [18]. Another approach was introduced in [19], using fuzzy logic to optimize the weights of the objective function. An improved design to improve the robot's ability to avoid dynamic obstacles was proposed for swarm robots [20]. Alternatively, some global path planning algorithms, such

TABLE 3. Results of the manually tuned classical DWA in an opening environment.

Weights			Time (seconds)	Average Velocity (m/s)	Average Path Length (m)
α	β	γ			
0.1	0.2	0.3	80.20	0.94	75.29
0.1	0.3	0.6	46.69	0.78	36.33
0.5	0.2	0.3		failed to reach the target	
0.1	0.2	0.1	31.73	0.51	16.10
0.3	0.1	0.4		failed to reach the target	
0.3	0.3	0.3	36.99	0.67	24.76

TABLE 4. Optimized weights for PSO-DWA-OFF.

Training Environment	α	β	γ
Corridor	0.4388	0.5612	1.0000
Opening	0.7896	0.2012	0.8908
U-Shape	0.5996	0.3549	0.6718

as A*, were combined with DWA, which serves as a local planner [21]. In the same way, the same method of integration between A* and DWA can be used for Unmanned Surface Vehicles [22]. Similarly, another approach was taken using the DWA-based adaptive Stanley control algorithm, also known as (DWA-Stanley) [23]. Improved Dynamic Window Approach (I-DWA) with Lyapunov stability criteria was used to guarantee global and asymptotic convergence to the goal while avoiding collisions [24].

Over the years, several attempts have been made to solve the limitations of the DWA and improve its performance as shown in Table 1. It has been adopted in different ways with a variety of levels of intelligence such as deep learning, reinforcement learning, and fuzzy logic. In addition, it was modified to adapt to different types of environments and platforms, as it can be implemented for AGVs, unmanned surface vehicles (USVs), Autonomous Underwater Vehicles (AUSs), and swarm robotics.

The main contribution of this paper is to develop an intelligent optimization framework using PSO inspired by swarm intelligence to perform online tuning of the objective function of the DWA. This enables efficient exploration of the solution space and optimization of robot navigation in a static environment while balancing goal navigation, obstacle avoidance, and trajectory smoothness. The PSO's global search capabilities can be utilized for offline DWA optimization, which alludes to calculating the optimum DWA weights before navigating the environment given some prior knowledge of the environment, which will remain constant during runtime. Alternatively, the DWA weights can also be continuously adjusted during the runtime of the DWA algorithm, referred to as the online approach. In both cases, the PSO iteratively evolves by minimizing a cost function that considers the distance to the goal, the smoothness of the trajectory, and the clearance from obstacles. As a result, the proposed frameworks try to improve the success rate, reduce

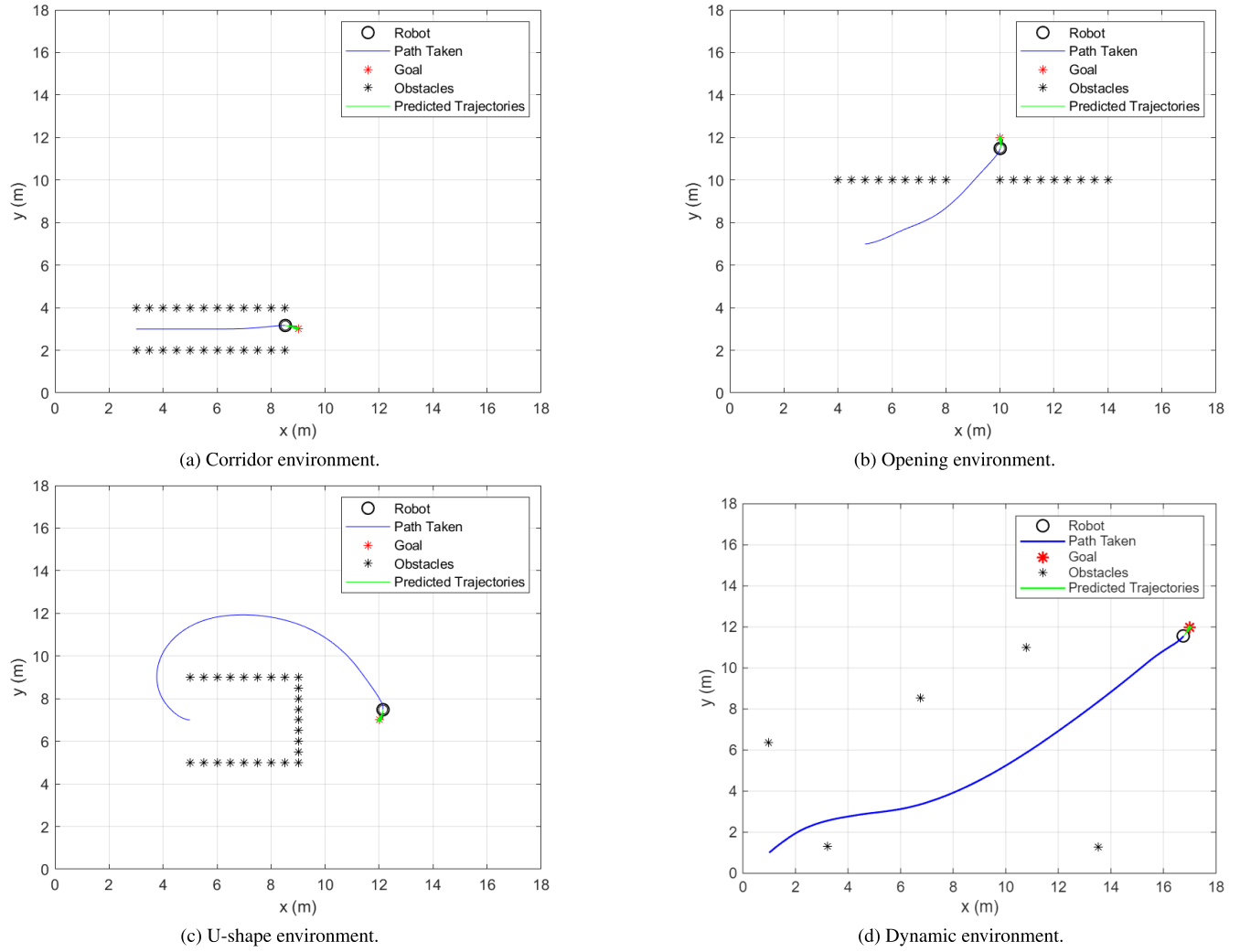


FIGURE 5. Simulation of the Classical DWA in four scenarios: (a) corridor, (b) opening, (c) U-shape, and (d) dynamic environment.

TABLE 5. Optimized weights for GA-DWA-OFF.

Training Environment	α	β	γ
Corridor	0.455	0.545	0.960
Opening	0.735	0.245	0.825
U-Shape	0.572	0.368	0.650

travel time and path length, increases average velocity, and ensures collision-free navigation.

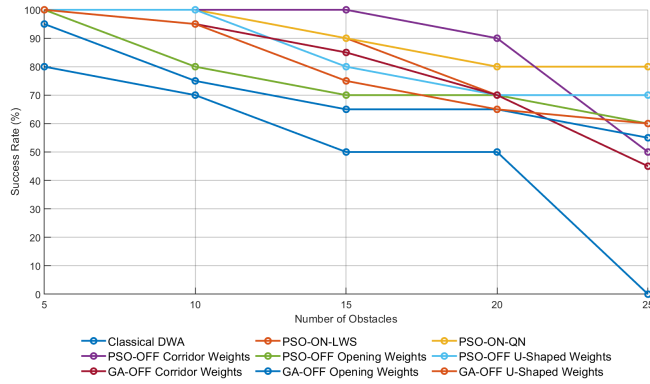
The PSO was selected as it requires a smaller number of hyperparameters than Genetic Algorithms (GA), Simulated Annealing (SA) and CMA-ES as shown in Table 2. In addition, it has a low computational overhead compared to model-based methods like Bayesian optimization [26], which can become expensive in high-dimensional spaces. PSO provides a well-controlled balance between global search (exploration) and fine-tuning (exploitation) through its inertia weight and cognitive/social learning coefficients [27]. This reduces the probability of getting stuck in local minima,

especially in rugged or multi-modal search spaces [28]. The swarm-based nature of PSO allows multiple candidate solutions to be evaluated simultaneously, which is well suited for parallel hardware and simulation environments [29].

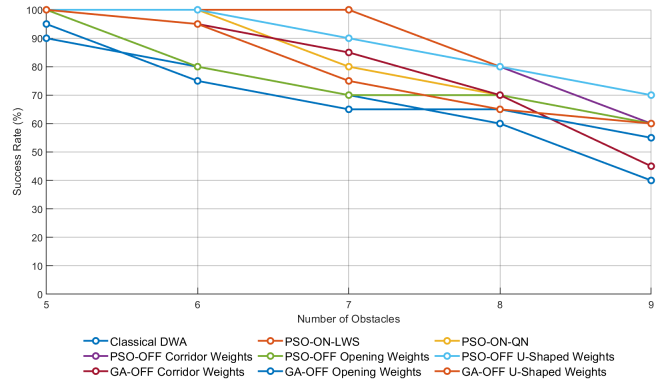
In Section II, the main principles of DWA are introduced. Section III describes the proposed framework for adaptive DWA, where the PSO algorithm is integrated with DWA. The proposed framework is tested in both simulated and experimental environments, and the results are discussed in Section IV. Finally, the concluding remarks are presented in Section V.

II. DYNAMIC WINDOW APPROACH

The main principle of the DWA as described by D. Fox et al. [5] is to filter all velocity commands that lead to collision with obstacles, considering the kinematic constraints of the robot. The rest of the velocities are the ones feasible within the kinematic constraints, also known as admissible velocities. This process defines a search space for



(a) PSO-DWA-ON, PSO-DWA-OFF, and Classical DWA static random simulated environment success rate vs. number of obstacles.



(b) PSO-DWA-ON, PSO-DWA-OFF, and Classical DWA dynamic random simulated environment success rate vs. number of obstacles.

FIGURE 6. Success rate comparison in (a) static and (b) dynamic random environments.

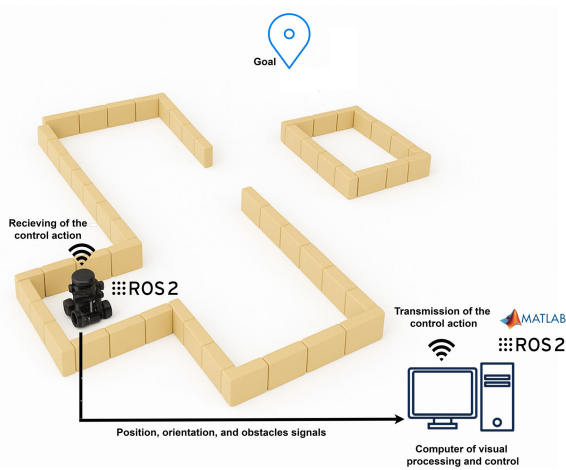


FIGURE 7. Diagram of the experimental platform configuration for the TurtleBot3 Burger mobile robot control.

selecting the next motion commands, ensuring the progression of the robot towards the goal while avoiding obstacles. Once the search area is established the optimal velocity is determined based on three key criteria: target alignment, obstacle clearance, and velocity optimization. These three criteria form the basis for the objective function [19]. The complete evaluation process of the DWA objective function is illustrated in Algorithm 1.

$$\text{Score}(v, \omega) = \alpha G(v, \omega) + \beta O(v, \omega) + \gamma V(v, \omega) \quad (1)$$

where G is a function that represents the alignment of the robot with the goal, which is the angle between the robot heading angle and the goal position (g_x, g_y) denoted as θ , and can be calculated as follows:

$$\theta = \psi - \phi \quad (2)$$

$$\psi = \tan^{-1} \left(\frac{g_x}{g_y} \right) \quad (3)$$

where θ is the target heading, ϕ is the robot heading angle, and ψ is the goal angle with respect to the global reference

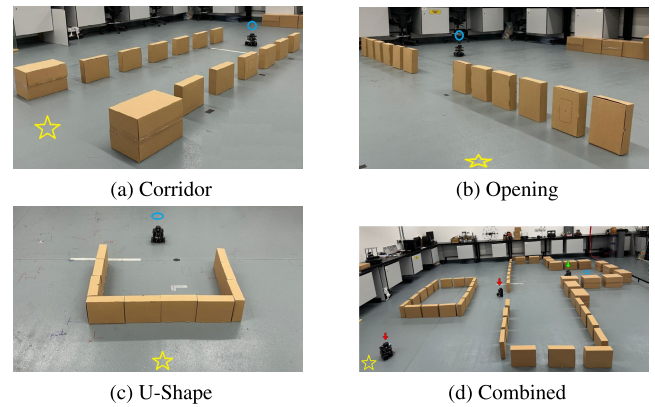


FIGURE 8. Laboratory test environments for validating PSO-DWA using TurtleBot3 platform.

frame. Furthermore, to normalize the target heading into the desired heading relative to the robot's forward direction, the transformation in Equation 4 is applied. So, the robot is aligned with the convention of the goal coordinate system as shown in Figure 2 [19].

$$\text{heading} = 180^\circ - \theta \quad (4)$$

In the objective function, O is a function that represents the distance between the robot position and the nearest obstacle d_{obs} . This is the distance between the robot position (x, y) and the closest obstacle position (d_x, d_y), both expressed with respect to the global reference frame, as illustrated in Figure 2 and equation (5). For obstacles of irregular shapes, the distance is calculated from the position of the robot to the nearest vertex of the obstacle defined by the obstacle space. The last part of the objective function, V , is a function that represents the linear velocity of the robot.

$$d_{obs} = \sqrt{(d_x - x)^2 + (d_y - y)^2} \quad (5)$$

The DWA relies on the dynamic constraints of the robot to define the set of admissible velocities. The motion of a mobile

TABLE 6. Static environments: Classical DWA vs. GA-DWA-OFF and PSO-DWA-OFF (pre-optimized on Corridor, Opening, and U-Shape).

Classical DWA					GA-DWA-OFF					PSO-DWA-OFF				
Environment	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)	Environment	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)	Environment	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)
Corridor	100%	42.50	0.31	13.08	Pre-optimized: Corridor					Pre-optimized: Corridor				
Opening	100%	47.02	0.78	36.87	Corridor	100%	22.70	0.59	13.90	Corridor	100%	21.43	0.61	13.17
U-Shape	100%	37.38	0.75	27.93	Opening	100%	44.20	0.84	37.80	Opening	100%	43.25	0.86	37.03
Combination	0%	failed to reach the target			U-Shape	0%	failed to reach the target			U-Shape	0%	failed to reach the target		
					Combination	0%	failed to reach the target			Combination	0%	failed to reach the target		
					Pre-optimized: Opening					Pre-optimized: Opening				
					Corridor	100%	26.50	0.52	8.90	Corridor	100%	24.33	0.54	7.83
					Opening	100%	45.20	0.45	22.50	Opening	100%	44.64	0.47	21.35
					U-Shape	100%	36.20	0.70	25.70	U-Shape	100%	34.89	0.71	24.90
					Combination	100%	39.70	0.72	29.20	Combination	100%	38.54	0.74	28.33
					Pre-optimized: U-Shape					Pre-optimized: U-Shape				
					Corridor	100%	30.90	0.71	21.60	Corridor	100%	30.16	0.72	20.98
					Opening	100%	26.40	0.56	15.10	Opening	100%	25.59	0.57	14.54
					U-Shape	100%	33.70	0.74	24.90	U-Shape	100%	32.50	0.75	24.26
					Combination	100%	36.10	0.77	28.30	Combination	100%	34.87	0.79	27.61

TABLE 7. Static random environments (5–25 static obstacles): Classical DWA vs. GA-DWA-OFF and PSO-DWA-OFF.

Classical DWA					GA-DWA-OFF					PSO-DWA-OFF				
# Obs.	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)	# Obs.	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)	# Obs.	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)
5	80%	29.01	0.69	20.01	<i>Pre-optimized: Corridor</i>					<i>Pre-optimized: Corridor</i>				
10	70%	79.55	0.40	31.82	5	100%	44.80	0.80	36.20	5	100%	43.92	0.81	35.54
15	50%	69.63	0.38	26.46	10	95%	42.90	0.80	33.90	10	100%	40.61	0.81	32.84
20	50%	61.80	0.45	27.81	15	85%	45.60	0.79	35.10	15	100%	41.46	0.81	33.56
25	0%	<i>failed to reach the target</i>			20	70%	48.10	0.77	32.90	20	90%	41.62	0.81	30.27
					25	45%	52.00	0.76	36.50	25	50%	43.06	0.81	34.77
					<i>Pre-optimized: Opening</i>					<i>Pre-optimized: Opening</i>				
					5	95%	44.20	0.80	35.50	5	100%	43.15	0.81	35.03
					10	75%	49.90	0.80	39.50	10	80%	47.86	0.81	38.98
					15	65%	50.90	0.79	40.00	15	70%	47.97	0.82	39.41
					20	65%	55.60	0.75	41.80	20	70%	53.48	0.77	41.25
					25	55%	56.90	0.74	42.30	25	60%	53.80	0.77	41.09
					<i>Pre-optimized: U-Shape</i>					<i>Pre-optimized: U-Shape</i>				
					5	100%	43.60	0.80	35.10	5	100%	42.22	0.81	34.30
					10	95%	50.20	0.70	35.60	10	100%	48.86	0.71	34.68
					15	75%	52.90	0.69	36.10	15	80%	50.22	0.70	35.21
					20	65%	57.60	0.72	41.10	20	70%	55.15	0.73	40.06
					25	60%	58.70	0.73	42.10	25	70%	56.01	0.74	41.49

robot can be expressed by the following kinematic model:

$$\dot{x}(t) = v(t) \cos(\phi(t)) \quad (6)$$

$$\dot{y}(t) = v(t) \sin(\phi(t)) \quad (7)$$

$$\dot{\phi}(t) = \omega(t) \quad (8)$$

where $(x(t), y(t))$ is the position of the robot in the global frame, $\phi(t)$ is the robot orientation, $v(t)$ is the linear velocity, and $\omega(t)$ is the angular velocity. These equations describe the instantaneous evolution of the robot pose under the control inputs $v(t)$ and $\omega(t)$. For compactness, the robot state can be defined as:

$$\xi(t) = [x(t) \ y(t) \ \phi(t) \ v(t) \ \omega(t)]^T, \quad (9)$$

To ensure feasibility, the DWA constrains the velocities to the intervals:

$$v \in [v_{\min}, v_{\max}], \quad \omega \in [\omega_{\min}, \omega_{\max}] \quad (10)$$

Furthermore, acceleration limits are imposed to guarantee that the robot can physically achieve the commanded velocities within the sampling interval Δt :

$$v \in [v(t) - \dot{v}_{\max} \Delta t, v(t) + \dot{v}_{\max} \Delta t] \quad (11)$$

$$\omega \in [\omega(t) - \dot{\omega}_{\max} \Delta t, \omega(t) + \dot{\omega}_{\max} \Delta t] \quad (12)$$

These constraints define the dynamic window, which represents the subset of admissible velocity pairs (v, ω) that satisfy both the kinematic limits and the dynamic capabilities

TABLE 8. Dynamic random environments (5–9 moving obstacles): Classical DWA vs. GA-DWA-OFF and PSO-DWA-OFF (pre-optimized on Corridor, Opening, and U-Shape).

Classical DWA					GA-DWA-OFF					PSO-DWA-OFF				
# Obs.	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)	# Obs.	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)	# Obs.	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)
5	90%	44.61	0.81	28.74	<i>Pre-optimized: Corridor</i>					<i>Pre-optimized: Corridor</i>				
6	80%	49.66	0.81	40.45	5	100%	45.90	0.81	37.50	5	100%	44.66	0.82	36.75
7	70%	44.23	0.81	25.07	6	95%	49.80	0.81	39.90	6	100%	47.51	0.82	38.86
8	60%	44.16	0.81	21.39	7	85%	52.70	0.79	41.70	7	90%	50.33	0.80	40.38
9	40%	50.36	0.81	16.33	8	70%	58.90	0.75	44.40	8	80%	55.91	0.76	42.67
					9	50%	73.50	0.74	54.30	9	60%	70.16	0.75	52.34
					<i>Pre-optimized: Opening</i>					<i>Pre-optimized: Opening</i>				
					5	100%	46.00	0.80	36.00	5	100%	44.81	0.81	36.31
					6	90%	50.00	0.80	38.00	6	90%	47.57	0.81	38.46
					7	80%	58.50	0.79	41.00	7	90%	56.33	0.76	42.90
					8	65%	69.00	0.76	49.00	8	70%	67.11	0.73	48.97
					9	55%	75.00	0.74	53.00	9	60%	73.70	0.71	52.06
					<i>Pre-optimized: U-Shape</i>					<i>Pre-optimized: U-Shape</i>				
					5	100%	45.50	0.81	36.60	5	100%	44.29	0.82	36.12
					6	95%	47.20	0.80	37.60	6	100%	45.35	0.81	36.65
					7	85%	52.10	0.79	41.00	7	90%	50.37	0.80	40.12
					8	70%	58.80	0.75	43.90	8	80%	56.32	0.76	42.93
					9	60%	64.90	0.74	46.90	9	70%	61.33	0.75	45.97

TABLE 9. PSO-DWA-ON performance with linear weighted-sum (LWS) and quadratic-normalized (QN) fitness functions.

PSO-DWA-ON (LWS)					PSO-DWA-ON (QN)				
<i>Static environments</i>					<i>Static environments</i>				
Environment	Success Rate	Time (s)	Avg. Velocity (m/s)	Avg. Distance (m)	Environment	Success Rate	Time (s)	Avg. Velocity (m/s)	Avg. Distance (m)
Corridor	100%	55.18	0.43	23.78	Corridor	100%	25.67	0.69	17.79
Opening	100%	79.74	0.42	33.33	Opening	100%	63.44	0.81	51.58
U-Shape	100%	53.60	0.79	42.35	U-Shape	100%	56.36	0.75	42.28
Combination	80%	89.28	0.49	47.86	Combination	100%	47.56	0.67	36.13
<i>Static random environments</i>					<i>Static random environments</i>				
# Obstacles	Success Rate	Time (s)	Avg. Velocity (m/s)	Avg. Distance (m)	# Obstacles	Success Rate	Time (s)	Avg. Velocity (m/s)	Avg. Distance (m)
5	100%	85.33	0.69	58.36	5	100%	38.24	0.79	30.19
10	100%	81.76	0.69	53.48	10	100%	49.46	0.83	41.33
15	90%	101.85	0.56	53.54	15	90%	47.22	0.77	36.46
20	70%	124.60	0.55	69.21	20	80%	45.12	0.82	37.02
25	60%	92.58	0.56	50.25	25	80%	50.18	0.77	37.94
<i>Dynamic random environments</i>					<i>Dynamic random environments</i>				
# Obstacles	Success Rate	Time (s)	Avg. Velocity (m/s)	Avg. Distance (m)	# Obstacles	Success Rate	Time (s)	Avg. Velocity (m/s)	Avg. Distance (m)
5	100%	36.45	0.76	27.80	5	100%	60.07	0.88	52.86
6	100%	65.06	0.68	44.30	6	100%	67.00	0.86	57.70
7	100%	83.49	0.66	55.29	7	80%	64.44	0.87	44.93
8	80%	83.90	0.70	58.50	8	70%	81.65	0.87	43.04
9	70%	77.32	0.68	52.80	9	60%	75.93	0.86	39.29

of the robot. The window is then searched to find the optimal control input according to the objective function in Equation (1).

III. ADAPTIVE DYNAMIC WINDOW APPROACH

To enable efficient exploration of the solution space and optimization of robot navigation in a static environment while balancing goal navigation, obstacle avoidance, and trajectory

smoothness, online tuning of the objective function of the DWA must be performed. PSO is one of the most well-known population-based metaheuristic optimization techniques [34], primarily due to its mathematical simplicity, relatively low parameterization, and strong convergence behavior across multimodal and high-dimensional optimization problems [35]. The algorithm architecture is based primarily on an iterative process. Starting with the initialization of a swarm

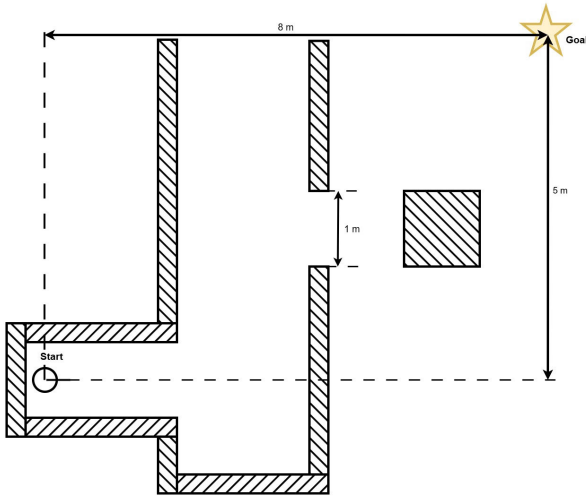


FIGURE 9. Combined environment map.

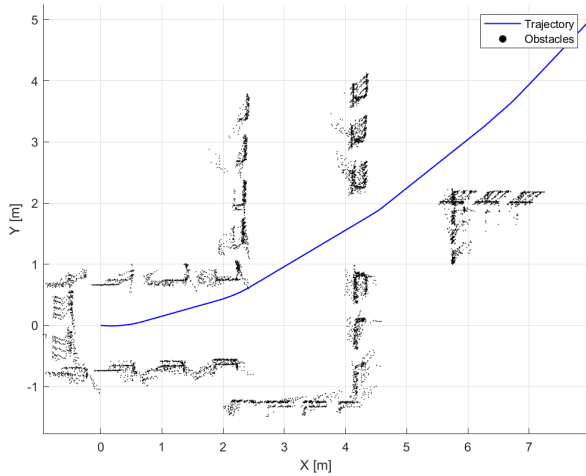


FIGURE 10. Robot trajectory and static obstacles using PSO-DWA-OFF opening pre-optimized weights in the combined environment.

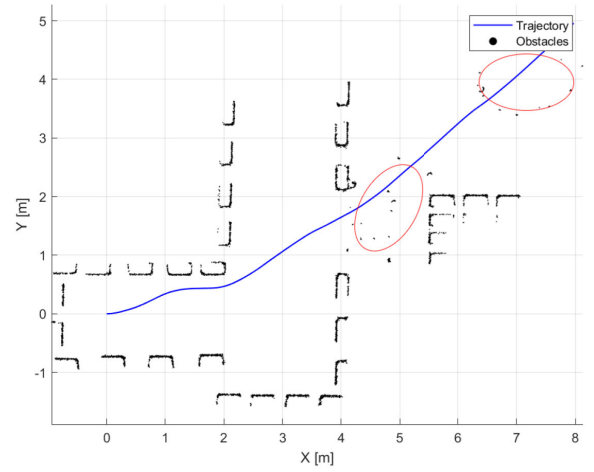


FIGURE 11. Robot trajectory and dynamic obstacles using PSO-DWA-OFF in random dynamic environment with U-Shape pre-optimized weights.

At first, these values are initialized to be random. With every iteration, the values are updated as follows [25]:

$$V_k(i+1) = \omega_p V_k(i) + c_1 r_1 (p_{\text{best},i}^k - X_k(i)) + c_2 r_2 (g_{\text{best},i} - X_k(i)) \quad (15)$$

$$X_k(i+1) = X_k(i) + V_k(i+1) \quad (16)$$

where $V_k(i+1)$ represents the velocity of particle k at iteration $i+1$, and $X_k(i)$ denotes the position of particle k at iteration i . The term $p_{\text{best},i}^k$ refers to the individual best position of particle k at iteration i , while $g_{\text{best},i}$ represents the global best position among all particles in iteration i . The coefficients c_1 and c_2 are acceleration coefficients, and r_1 and r_2 are uniform random numbers in the range $[0, 1]$. Lastly, ω_p is the adaptive inertia weight.

Two frameworks can be used to find the optimum parameters. Firstly, the offline framework, which starts by initializing the PSO parameters, position and velocity of the particles, as well as the goal pose and the local map, as shown in Algorithm 2. Next, the algorithm starts the iterative process to evaluate the fitness function and keeps updating the velocity and position of the particles. Once the stopping criteria is satisfied, the optimum parameters (α^* , β^* , γ^*) are plugged into the classical DWA and the robot starts navigating through the environment. Second, is the online framework, which recursively tries to find an optimal solution and update the DWA parameters (α_t , β_t , γ_t) during robot navigation, as shown in Figure 4 and detailed in Algorithm 3. The proposed framework uses the Linear Weighted Sum fitness function for evaluation. As shown in Equation (17), is a combination that maintains a direct relationship between the weighted criteria. The subtraction of the velocity term implies that higher velocities are penalized, promoting safer and more controlled high-speed navigation and enhancing stability.

$$\text{cost}_{\text{LWS}} = \alpha \cdot G(v, \omega) + \beta \cdot O(v, \omega) - \gamma \cdot V(v, \omega) \quad (17)$$

of particles, each representing a potential solution within the search space. As shown in Figure 3, as the algorithm progresses, the velocity and position of each particle are updated based on two key factors. First, is the personal best $p_{\text{best},i}^k$, which translates to the best objective function value that a particle has achieved individually. On the other hand, the second key factor is the global best $g_{\text{best},i}$ which represents the best solution found throughout the swarm. Throughout the continuous iterative process, the particles tune their positions by balancing exploration and exploitation. Finally, the swarm converges towards a near optimal solution with the lowest value of the fitness function.

The positions of the particles are represented by [27]:

$$P_i^t = [x_{0,i}^t, x_{1,i}^t, x_{2,i}^t, \dots, x_{n,i}^t] \quad (13)$$

The velocity of the particles is represented by:

$$V_i^t = [v_{0,i}^t, v_{1,i}^t, v_{2,i}^t, \dots, v_{n,i}^t] \quad (14)$$

TABLE 10. PSO-DWA-OFF experimental results in static environments using pre-optimized weights (Corridor, Opening, U-Shape).

Pre-optimized: Corridor					Pre-optimized: Opening					Pre-optimized: U-Shape				
Environment	Success Rate	Time (s)	Avg. Velocity (m/s)	Avg. Distance (m)	Environment	Success Rate	Time (s)	Avg. Velocity (m/s)	Avg. Distance (m)	Environment	Success Rate	Time (s)	Avg. Velocity (m/s)	Avg. Distance (m)
Corridor	100%	27.03	0.17	3.97	Corridor	100%	30.86	0.14	3.92	Corridor	100%	27.63	0.15	3.99
Opening	100%	30.94	0.18	5.70	Opening	100%	31.26	0.19	5.78	Opening	100%	32.83	0.17	5.80
U-Shape	0%	failed to reach the target			U-Shape	100%	33.23	0.18	5.89	U-Shape	100%	33.19	0.18	6.09
Combination	0%	failed to reach the target			Combination	100%	52.07	0.18	9.60	Combination	100%	53.41	0.18	9.84

TABLE 11. Experiment results of PSO-DWA-OFF in dynamic random environments using pre-optimized weights.

Pre-optimized Weights	Success Rate	Time (Seconds)	Average Velocity (m/s)	Average Distance (m)
Corridor	0%	failed to reach the target		
Opening	0%	failed to reach the target		
U-Shape	90%	90.54	0.17	9.59

IV. SIMULATION & EXPERIMENTAL RESULTS

The PSO-enhanced DWA (PSO-DWA) was compared with the classical DWA and a genetic algorithm (GA) offline tuned variant (GA-DWA-OFF). Two types of frameworks were implemented and tested, one with an online framework (PSO-DWA-ON), the other with an offline framework (PSO-DWA-OFF). The three frameworks are tested in a *MATLAB* simulation environment as well as experimentally in the lab using the *TurtleBot3 Burger* mobile robot. The evaluation process was carried out in three stages. First, the four defined static environments Corridor, Opening, U-Shape, and Combination, each tested under every pre-optimized weight set. Next, static random fields with 5, 10, 15, 20, and 25 obstacles using the same weights. Finally, dynamic random environments with 5 to 9 dynamic obstacles with a step increment of 1, again using the same weights. These frameworks are compared on the basis of the path length, the average linear velocity of the robot, and the time required to reach the goal. In addition, to calculate the success rate, each experiment was conducted 10 times to ensure the consistency of the system and the results.

A. SIMULATION RESULTS

1) CLASSICAL DWA SIMULATION RESULTS

The classical DWA was first tested in a simple opening environment, where different weights were randomly selected to find the optimized path as shown in Figure 5b. The robot starts its motion at position $\mathbf{p}_{\text{start}} = [5, 7]^T$ with an initial velocity of 0 m/s, while the goal is set to $\mathbf{p}_{\text{goal}} = [10, 12]^T$. Several manual tuning iterations were performed to find the optimal weights of [0.1, 0.2, 0.1], which led to the best average velocity, average path length and time, as shown in Table 3.

Then these optimal weights were tested in four different environments such as Corridor, Opening, U-Shape, and Combination as shown in Figures 5a, 5b, and 5c. The success

rate was added to the evaluation metrics along with the average time, velocity, and distance taken. As shown in Table 6, the robot can reach the goal in all environments except the Combination environment. Using the optimum weights selected from the opening environment, the classical DWA was tested in another set of static environments where the obstacles were placed at random positions. The main objective was to investigate the correlation between the number of obstacles and the success rate when using these optimum weights, as illustrated in Figure 6a. As presented in Table 6, the success rate decreases from 80% to 0% as the number of obstacles increases from 5 obstacles to 25 obstacles. Similarly, as shown in Figure 5d, the baseline classical DWA was tested in dynamic random environments to assess the effect of increasing the number of dynamic obstacles on the success rate, as shown in Figure 6b. These dynamic obstacles are modeled with a 2D random walk model. At each time step, their positions are updated by adding small random displacements Δx_k and Δy_k to the current coordinates, as shown in Equations 18, and 19.

$$x_o[k+1] = x_o[k] + \Delta x_k \quad (18)$$

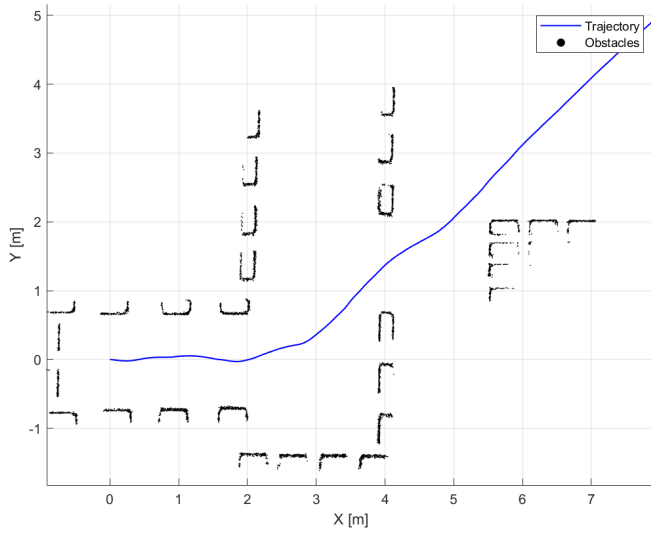
$$y_o[k+1] = y_o[k] + \Delta y_k \quad (19)$$

where $\mathbf{w}_k = [\Delta x_k, \Delta y_k]^T$ is a random vector with components sampled uniformly from the interval $[-0.5, 0.5]$. This produces jittery, unpredictable dynamic motion in the environment.

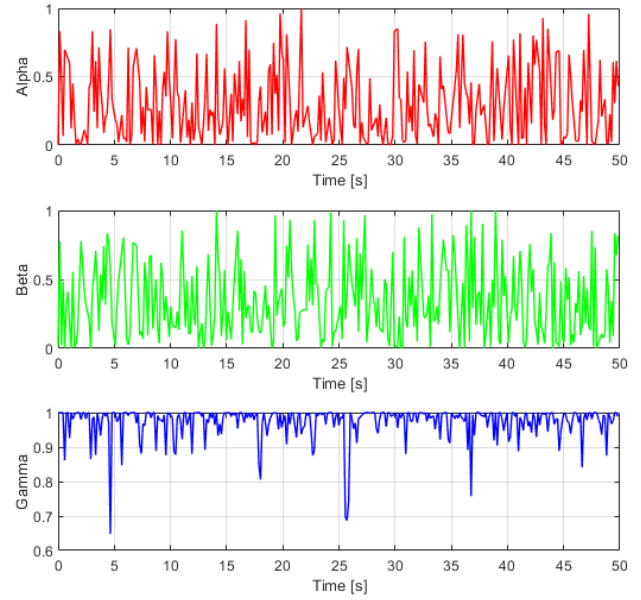
Consequently, as the number of obstacles increased from 5 obstacles to 9 obstacles, the success rate decreased from 90% to 40%, respectively. Furthermore, in both static and dynamic random environments, as the number of obstacles increases, the travel time increases, and the average velocity drops.

2) PSO-DWA-OFF SIMULATION RESULTS

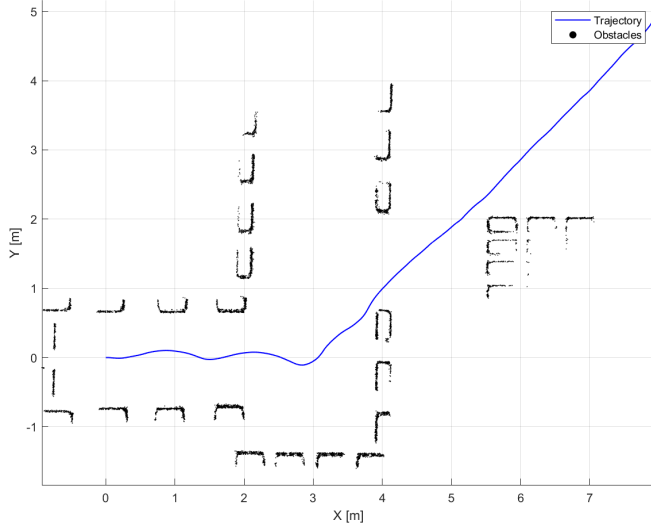
In the offline PSO-DWA, the PSO algorithm is used to optimize the weights offline until the optimal values are found. In addition, the algorithm was configured with a swarm size of 10 particles and 100 iterations. The cognitive coefficient c_1 and the social coefficient c_2 were both set to 1.49, the inertia range was set to $\omega_p \in [\omega_{p,\min}, \omega_{p,\max}] = [0.1, 1.1]$, and the tolerance was set to 10^{-6} . This configuration provides a good trade-off between convergence speed and computation time. These optimized weights are then utilized with the classical DWA algorithm. Weight optimization is carried



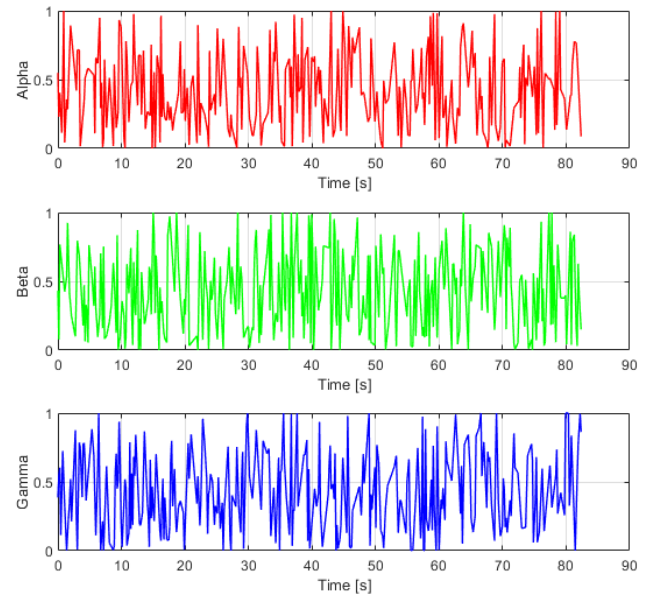
(a) Robot trajectory and obstacles using PSO-DWA-ON with linear weighted-sum fitness.



(b) Optimized parameters of PSO-DWA-ON with linear weighted-sum fitness.



(c) Robot trajectory and obstacles using PSO-DWA-ON with quadratic normalized fitness.



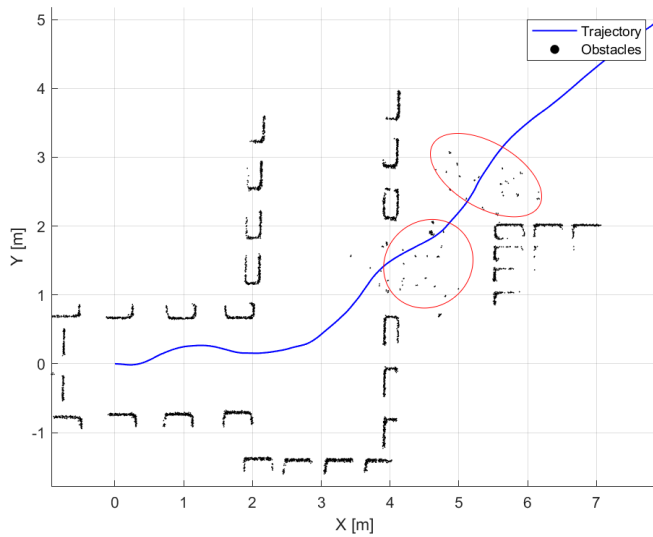
(d) Optimized parameters of PSO-DWA-ON with quadratic normalized fitness.

FIGURE 12. PSO-DWA-ON in the combined static environment: (a–b) linear weighted-sum (LWS) fitness; (c–d) quadratic normalized (QN) fitness.

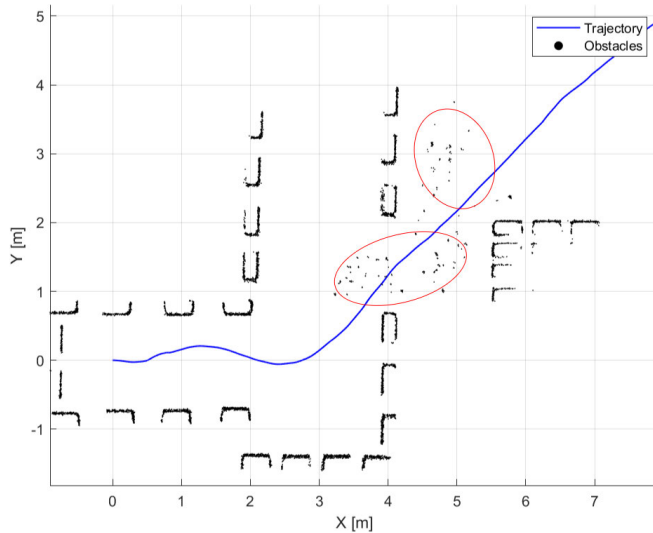
out using the offline PSO-DWA and testing environments listed in Table 4. After conducted optimization process, the optimized weights are evaluated in the same environments. To ensure a valid evaluation of the PSO-DWA-OFF and to fully exploit its performance, a GA-DWA-OFF framework was also developed to generate adaptive weights in the same manner. The GA parameters are configured as follows: a population size of 30 individuals is used, with a chromosome length of 3 corresponding to the three weights. The algorithm

runs for 50 generations. Tournament selection is applied to choose parents for reproduction. The crossover probability is set to 0.8, while the mutation probability is 0.05, ensuring sufficient exploration of the search space. Additionally, elitism is used to carry over the top 2 individuals to the next generation, preserving the best solutions.

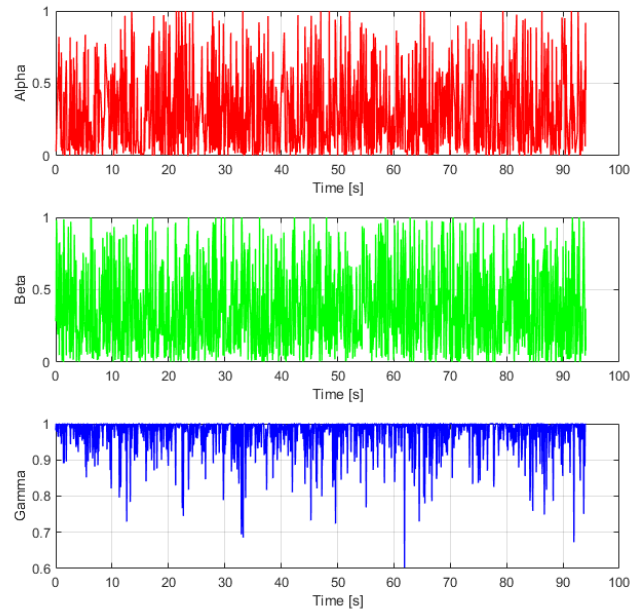
First, a genetic algorithm in the GA-DWA-OFF framework is used to generate the optimized weights offline shown in Table 5 and then used with the classical DWA. As shown in



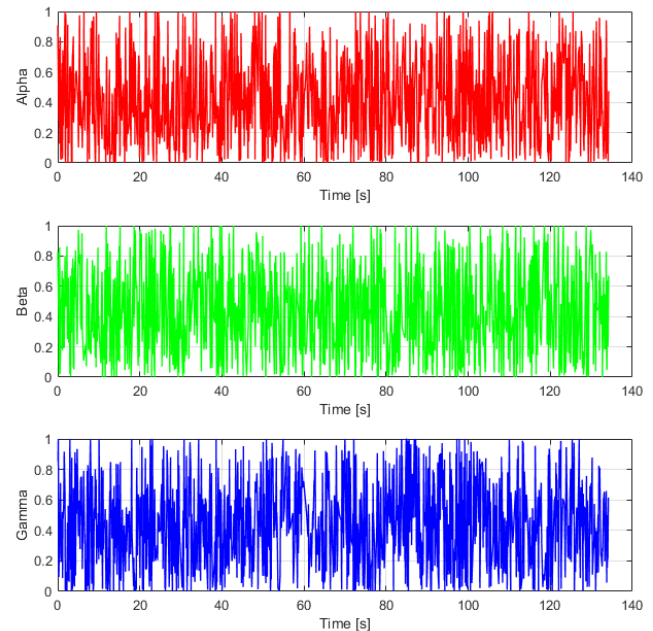
(a) Robot trajectory and obstacles using PSO-DWA-ON (linear weighted-sum fitness).



(c) Robot trajectory and obstacles using PSO-DWA-ON (quadratic normalized fitness).



(b) Optimized parameters of PSO-DWA-ON (linear weighted-sum fitness).



(d) Optimized parameters of PSO-DWA-ON (quadratic normalized fitness).

FIGURE 13. PSO-DWA-ON in a random dynamic environment: (a–b) LWS; (c–d) QN.

Table 6, optimized weights with the Corridor environment led to a 100% success rate in both the Corridor and the Opening environments, but the algorithm did not reach the goal in any other environment. However, optimization with opening and U-Shape environments achieved both a 100% success rate in all test cases. Generally, optimization with the U-Shape environment had the highest average velocities in all test environments. Similarly, in static and dynamic random

environments, GA-DWA improved the success rates in all test environments. As shown in Tables 7 and 8, the U-Shape weights had the highest success rate at the maximum number of obstacles in static and dynamic random environments of 60%, exceeding the corridor and opening pre-optimized weights of 45% and 55%, respectively.

Consistently, in the PSO-DWA-OFF, optimization with the Corridor environment yielded a 100% success rate in

TABLE 12. PSO-DWA-ON: experimental results Comparison of Linear Weighted-Sum vs. Quadratic Normalized fitness functions.

Static environments								
Environment	Linear Weighted-Sum				Quadratic Normalized			
	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)
Corridor	100%	22.19	0.19	4.08	100%	35.31	0.12	3.92
Opening	100%	33.59	0.19	3.76	100%	64.67	0.15	3.95
U-Shape	100%	38.36	0.21	4.61	100%	35.31	0.12	3.32
Combination	100%	49.83	0.21	7.83	100%	82.35	0.15	8.59

Dynamic random environments				
Fitness Function	Success Rate	Time (s)	Avg. Vel. (m/s)	Avg. Dist. (m)
Linear Weighted-Sum	100%	94.06	0.19	9.96
Quadratic Normalized	100%	134.20	0.14	8.78

TABLE 13. Experimental results showing computation time comparison between Classical DWA and PSO-DWA-ON.

Method	Time (ms)	Real-Time Suitability
Classical DWA	15	Suitable for 30–50 Hz control loop
PSO-DWA-ON	48	Meets 20 Hz real-time constraint

both the Corridor and the Opening environments, but the algorithm failed to reach the goal in the remaining test cases. Additionally, using weights optimized in opening and U-Shape environments resulted in a 100% success rate in all environments. Between the two, the U-Shape optimized weights achieved a higher average velocity. However, in terms of completion time, the opening optimized weights performed better in the Corridor environment only. In static random environments, PSO-DWA-OFF using pre-optimized U-Shape weights achieved a 10% higher success rate than GA-DWA-OFF at the maximum obstacle count (25 obstacles). In other words, the rate of decay of the success rate with the increase of the number of obstacles of the PSO-DWA-OFF is lower than that of the classical DWA, as shown in Figure 6a. As shown in Table 7, the success rates of the U-Shape pre-optimized weights are higher than those of the Corridor and Opening pre-optimized weights. Similarly, simulation of the random dynamic environment resulted in the same increase in success rates for the U-Shape pre-optimized weights. Generally, the PSO-DWA-OFF shows the same trend as GA-DWA-OFF that of having a longer travel time and a larger average velocity than the classical DWA. Nevertheless, GA-DWA-OFF has the longest travel time in all cases, yet PSO-DWA-OFF achieved the highest average velocity.

3) PSO-DWA-ON SIMULATION RESULTS

The PSO-DWA-ON framework was tested in the same four predefined environments: Corridor, Opening, U-Shape, and

Combination. Two fitness functions were tested with PSO-DWA-ON. The first algorithm uses a Linear Weighted Sum fitness function, referred to as PSO-LWS as in Equation 17, while the second uses a Quadratic Normalized fitness function, referred to as PSO-QN. As the name suggests, they differ from each other in terms of linearity. In addition, the quadratic normalized shown in Equation (20), is more sensitive to large deviation [34]. Both frameworks achieved the same success rate in corridor, opening, and U-Shape environments. However, the average velocity of the PSO-QN function was higher than that of the PSO-LWS, resulting in a shorter completion time. When tested in the combined environment, both fitness functions outperformed GA-DWA-OFF, PSO-DWA-OFF, and classical DWA. As shown in Table 9, the PSO-QN had a 100% success rate compared to 80% for PSO-LWS in the combined environment while maintaining a higher average velocity and a shorter travel time.

$$\text{cost}_{\text{QR}} = \frac{(\alpha \cdot G(v, \omega) + \beta \cdot O(v, \omega) + \gamma \cdot V(v, \omega))^2}{\alpha - \beta - \gamma} \quad (20)$$

Furthermore, evaluate the robustness of both fitness functions, additional experiments were conducted in random static and dynamic environments. The same frameworks were tested on random static environments with varying numbers of obstacles. As shown in Figure 6a, both algorithms exhibit a similar trend in terms of success rate in environments with low obstacle density. However, the average velocity of the PSO-QN was higher than that of the PSO-LWS, resulting in a shorter completion time, as shown in Table 9. In addition, PSO-QN achieved the highest success rate of 80% in the environment of random obstacle with the highest density compared to PSO-LWS of 60% and PSO-DWA-OFF with pre-optimized U-Shape weights of 70%. In terms of random dynamic environments, the PSO-LWS outperformed PSO-QN and PSO-DWA-OFF with pre-optimized U-Shape weights in terms of success rate, but with a slight compromise in travel time and average velocity, as shown in Figure 6b.

B. EXPERIMENTAL RESULTS

1) EXPERIMENTAL SETUP

The proposed adaptation of the DWA using PSO optimization techniques was experimentally validated using the TurtleBot3 Burger mobile robot platform, which is a differential-drive wheeled robot equipped with a 360° LiDAR sensor capable of detecting objects up to 3.5 meters away. The robot utilizes the odometry of its wheel encoders for localization. The experimental setup is shown in Figure 7, the robot's low level control was executed on-board using ROS 2 Humble, while the proposed framework was deployed on an off-board computer connected to the same network for real-time communication. As shown in Figure 8, there are four laboratory test environments used to carry out the experiments where the start position is indicated by a blue circle and the goal pose by a yellow star. The environments consisted of static obstacles and two mobile robots acting as dynamic obstacles, indicated by red arrows, in which their paths are modeled using Eqs. (21), (22), and (23). The robot that uses the DWA framework is indicated by a green arrow in Figure 8d.

$$\theta_o(t) = \theta_o + s \omega t \quad (21)$$

$$x_o(t) = c_x + R \cos(\theta_o(t)) \quad (22)$$

$$y_o(t) = c_y + R \sin(\theta_o(t)) \quad (23)$$

where $\theta_o(t)$ denotes the obstacle's angular position at time t , with θ_o as the initial angle. The parameter $s \in +1, -1$ specifies the rotation direction (+1 for counterclockwise, -1 for clockwise), and ω is the constant angular speed. The Cartesian position $x_o(t), y_o(t)$ is defined relative to the circle center (c_x, c_y) with radius R .

To exploit the effectiveness and success rates of the proposed frameworks, multiple environment configurations such as simulations were used to conduct the experiments, including corridor, opening, and U-Shape environments. In addition, a combined environment that incorporates all the characteristics of the other environments, as shown in Figure 9.

2) PSO-DWA-OFF EXPERIMENTAL RESULTS

The PSO-DWA-OFF framework was tested using the pre-optimized weights of each environment, as shown in Table 4. As shown in Table 10, the corridor pre-optimized weights achieve the highest average velocity of 0.17 m/s and a shortest travel time of 27.03 seconds in the corridor environment compared to the other pre-optimized weights. Although having 100% success rates in the corridor and opening environments, PSO-DWA-OFF fails to reach the goal in all trials in the U-Shape and combination environments using the pre-optimized weights of the corridor. Furthermore, in the opening environment, all pre-optimized weights are closely matched in terms of travel time and average velocity. However, the U-Shape pre-optimized weights have a slightly higher travel distance, and the opening weights have a slight advantage in terms of average velocity. Moreover, both the

opening and U-Shape pre-optimized weights have 100% success rates and the same average velocity. Nevertheless, the opening pre-optimized weights have a shorter travel time and distance as shown in Figure 10. Regarding the dynamic environment, the robot was able to reach the goal 90% of the tests without causing any collisions only using the pre-optimized weights of the U-shape environment, as shown in Table 11. As shown in Figure 11, dynamic obstacles move in a predefined circular path as shown in red traveling with constant velocity.

3) PSO-DWA-ON EXPERIMENTAL RESULTS

Furthermore, for the PSO-DWA-ON, the same static environment experiments were conducted twice using both PSO-LWS and PSO-QN. As shown in Table 12, the PSO-LWS leads to a shorter travel time of 22.19 seconds and a higher average velocity of 0.19 m/s compared to the PSO-DWA-OFF with corridor pre-optimized weights. However, in the opening environment the average distance traveled is significantly dropped, despite having a travel time similar to that of the PSO-DWA-OFF. As for the U-Shape and Combination environments, the travel time is less than PSO-DWA-OFF, smaller travel distance and larger average velocity. For PSO-QN, despite having a shorter travel distance in all environments, the average velocity decreased and the travel time increased. As shown in Figure 12, the robot's trajectories are fairly similar despite the difference in weights of both fitness functions. In terms of dynamic environment, both fitness functions showed a success rate of 100%. On the other hand, the PSO-LWS had a higher average velocity and shorter travel time than the PSO-QN. As shown in Figure 13, both fitness functions produced similar trajectories and successfully avoided dynamic obstacles following the predefined oval path shown in red.

In the proposed PSO-DWA-ON method, the planning cycle runs at 10 Hz (100 ms per step). The average computation time per cycle is 48 ms over 10 trials in dynamic scenarios, with a worst case of 68 ms. These results indicate that the system reliably satisfies the real-time constraint with no deadline violations observed under standard operating conditions. As shown in Table 13, the computation complexity of PSO-DWA-ON is around three times greater than the classical DWA. However, PSO-DWA-ON provided outstanding performance in terms of success rate, average velocity, travel time, and distance compared to classical DWA.

V. CONCLUSION

In conclusion, the problem of tuning the DWA parameters was addressed by the proposed optimization framework using PSO. The online and offline variants of the PSO-DWA framework were compared with the classical DWA and GA-DWA-OFF in static and dynamic environments. The offline approach demonstrated an improvement in performance over the classical DWA by 10% to 30% in success rates.

Similarly, a performance gap between the genetic algorithm and particle swarm optimization in the offline approach of 5% to 10% in terms of success rates justified the choice of using PSO in both frameworks. However, the online approaches outperformed PSO-DWA-OFF, GA-DWA-OFF, and classical DWA. In particular, the online PSO-DWA-LWS fitness function achieved a higher average velocity by 27%, a longer travel distance by 10%, and a shorter travel time by 30% compared to PSO-QN. This trend is attributed to the tendency of PSO-LWS to assign a higher weight on velocity during optimization. By analyzing the robot trajectories of both fitness functions, the LWS fitness function offers better maneuverability around dynamic obstacles. The results of the simulation and the experiment showed a consistent trend of increased success rate, increased average velocity, and decreased travel time. Furthermore, the online PSO-DWA-ON with a linear weighted sum fitness function demonstrated the strongest ability to adapt its parameters in any complex environment without compromising overall performance.

ACKNOWLEDGMENT

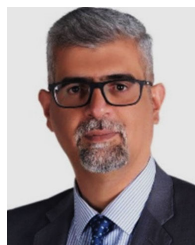
This article represents the opinions of the author(s) and does not mean to represent the position or opinions of American University of Sharjah.

REFERENCES

- [1] H. Haghighi, D. Delahaye, and D. Asadi, "Performance-based emergency landing trajectory planning applying meta-heuristic and Dubins paths," *Appl. Soft Comput.*, vol. 117, Mar. 2022, Art. no. 108453.
- [2] H. Haghighi, D. Delahaye, J.-M. Moschetta, and D. Asadi, "UAV trajectory optimization for maximum soaring in windy environment," *Unmanned Syst.*, vol. 13, no. 2, pp. 399–412, Mar. 2025.
- [3] J. Borenstein and Y. Koren, "The vector field histogram-fast obstacle avoidance for mobile robots," *IEEE Trans. Robot. Autom.*, vol. 7, no. 3, pp. 278–288, Jun. 1991.
- [4] N. Yong Ko and R. G. Simmons, "The lane-curvature method for local obstacle avoidance," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. Innov. Theory, Pract. Appl.*, Oct. 1998, pp. 1615–1621.
- [5] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robot. Autom. Mag.*, vol. 4, no. 1, pp. 23–33, Mar. 1997.
- [6] Y. Li, R. Jin, X. Xu, Y. Qian, H. Wang, S. Xu, and Z. Wang, "A mobile robot path planning algorithm based on improved A* algorithm and dynamic window approach," *IEEE Access*, vol. 10, pp. 57736–57747, 2022.
- [7] D. Shan, S. Zhang, X. Wang, and P. Zhang, "Path-planning strategy: Adaptive ant colony optimization combined with an enhanced dynamic window approach," *Electronics*, vol. 13, no. 5, p. 825, Feb. 2024.
- [8] J. R. Sánchez-Ibáñez, C. J. Pérez-Del-Pulgar, and A. García-Cerezo, "Path planning for autonomous mobile robots: A review," *Sensors*, vol. 21, no. 23, p. 7898, Nov. 2021.
- [9] L. Mengmeng, Y. Xiaofei, G. Jun, H. Jiabao, W. Qi, and W. Xu, "Dynamic obstacle avoidance method for unmanned surface vessel based on deep reinforcement learning and dynamic window approach," in *Proc. IEEE 13th Data Driven Control Learn. Syst. Conf. (DDCLS)*, May 2024, pp. 1069–1074.
- [10] X. Mai, D. Li, J. Ouyang, and Y. Luo, "An improved dynamic window approach for local trajectory planning in the environment with dense objects," *J. Phys., Conf. Ser.*, vol. 1884, no. 1, Apr. 2021, Art. no. 012003.
- [11] J. van den Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2008, pp. 1928–1935.
- [12] J. Zhang, Z. Zhu, Y. Xue, Z. Deng, and H. Qin, "Local path planning of under-actuated AUV based on VADWA considering dynamic model," *Ocean Eng.*, vol. 310, Oct. 2024, Art. no. 118705.
- [13] D. Jiang, L. Du, S. Li, M. Wang, H. Zhang, X. Chen, and Y. Sun, "An improved dynamic window approach based on reinforcement learning for the trajectory planning of automated guided vehicles," *IEEE Access*, vol. 12, pp. 36016–36025, 2024.
- [14] L. Chang, L. Shan, C. Jiang, and Y. Dai, "Reinforcement based mobile robot path planning with improved dynamic window approach in unknown environment," *Auto. Robots*, vol. 45, no. 1, pp. 51–76, Jan. 2021.
- [15] M. Kobayashi, H. Zushi, T. Nakamura, and N. Motoi, "Local path planning: Dynamic window approach with Q-learning considering congestion environments for mobile robot," *IEEE Access*, vol. 11, pp. 96733–96742, 2023.
- [16] M. Dobrevski and D. Skocaj, "Adaptive dynamic window approach for local navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 6930–6936.
- [17] P. Saranritichai, N. Niparnan, and A. Sudsang, "Robust local obstacle avoidance for mobile robot based on dynamic window approach," in *Proc. 10th Int. Conf. Electr. Eng./Electron., Comput., Telecommun. Inf. Technol.*, May 2013, pp. 1–4.
- [18] J. H. Zhang, Q. Feng, A. D. Zhao, W. He, and X. Hao, "Local path planning of mobile robot based on self-adaptive dynamic window approach," *J. Phys., Conf. Ser.*, vol. 1905, no. 1, May 2021, Art. no. 012019.
- [19] O. A. Abubakar, M. A. Jaradat, and M. F. Abdel-Hafez, "Intelligent optimization of adaptive dynamic window approach for mobile robot motion control using fuzzy logic," *IEEE Access*, vol. 10, pp. 119368–119378, 2022.
- [20] Y. Cao and N. Mohamad Nor, "An improved dynamic window approach algorithm for dynamic obstacle avoidance in mobile robot formation," *Decis. Anal. J.*, vol. 11, Jun. 2024, Art. no. 100471.
- [21] H. Song and D. Zhang, "Robot path planning algorithm based on improved A and DWA," *Frontiers Comput. Intell. Syst.*, vol. 6, no. 1, pp. 37–41, Nov. 2023, doi: 10.54097/fcis.v6i1.07.
- [22] Z. Chen, Y. Zhang, Y. Zhang, Y. Nie, J. Tang, and S. Zhu, "A hybrid path planning algorithm for unmanned surface vehicles in complex environment with dynamic obstacles," *IEEE Access*, vol. 7, pp. 126439–126449, 2019.
- [23] G. Li, P. Shang, C. Zheng, and D. Sun, "A lateral control method of intelligent vehicles based on shared control," *Symmetry*, vol. 14, no. 11, p. 2447, Nov. 2022.
- [24] H. Berti, Á. D. Sappa, and O. Agamennoni, "Improved dynamic window approach by using Lyapunov stability criteria," *Latin Amer. Appl. Res.*, vol. 38, no. 4, pp. 289–298, 2008.
- [25] Z. Yang, N. Li, Y. Zhang, and J. Li, "Mobile robot path planning based on improved particle swarm optimization and improved dynamic window approach," *J. Robot.*, vol. 2023, pp. 1–16, May 2023.
- [26] M. G. M. Hamami and Z. H. Ismail, "A systematic review on particle swarm optimization towards target search in the swarm robotics domain," *Arch. Comput. Methods Eng.*, vol. 30, pp. 1–20, Oct. 2022.
- [27] D. Freitas, L. G. Lopes, and F. Morgado-Dias, "Particle swarm optimisation: A historical review up to the current developments," *Entropy*, vol. 22, no. 3, p. 362, Mar. 2020.
- [28] M. A. Jaradat, L. S. Sawaqed, and M. M. Alzgoool, "Optimization of PID2-FLC for blood glucose level using particle swarm optimization with linearly decreasing weight," *Biomed. Signal Process. Control*, vol. 59, May 2020, Art. no. 101922.
- [29] M. I. Capel, A. Salguero-Hidalgo, and J. A. Holgado-Terriza, "Parallel PSO for efficient neural network training using GPGPU and apache spark in edge computing sets," *Algorithms*, vol. 17, no. 9, p. 378, Aug. 2024.
- [30] G. Nagib and W. Gharieb, "Path planning for a mobile robot using genetic algorithms," in *Proc. IEEE Robot.*, Sep. 2005, pp. 185–189.
- [31] H. Miao and Y.-C. Tian, "Robot path planning in dynamic environments using a simulated annealing based approach," in *Proc. 10th Int. Conf. Control, Autom., Robot. Vis.*, Dec. 2008, pp. 1253–1258.
- [32] N. Srinivas, A. Krause, S. M. Kakade, and M. Seeger, "Gaussian process optimization in the bandit setting: No regret and experimental design," 2009, arXiv:0912.3995.
- [33] N. Hansen, "The CMA evolution strategy: A tutorial," 2016, arXiv:1604.00772.
- [34] T. M. Shami, A. A. El-Saleh, M. Alswaitti, Q. Al-Tashi, M. A. Summakieh, and S. Mirjalili, "Particle swarm optimization: A comprehensive survey," *IEEE Access*, vol. 10, pp. 10031–10061, 2022.
- [35] J. Kennedy and R. C. Eberhart, "Particle swarm optimization," in *Proc. Int. Conf. Neural Netw.*, vol. 4, 2002, pp. 1942–1948.



SAMER S. SARKIS received the B.Sc. degree in mechanical engineering, in 2021. He is currently pursuing the M.Sc. degree with the Mechatronics Engineering Graduate Program, American University of Sharjah (AUS). His current research interests and experience include robotics, mobile robot dynamics, global and local path planning, and aerial manipulators.



MOHAMMAD A. JARADAT received the master's and Ph.D. degrees in mechanical engineering from Texas A&M University, College Station, TX, USA. He is currently with the Mechanical Engineering Department, American University of Sharjah (AUS) and Jordan University of Science and Technology (JUST). His scientific experience has resulted in many publications, patents, projects, and awards specialized. His research interests include robotics, artificial intelligent systems, sensor fusion, fault diagnostics, intelligent NANO-systems, intelligent control, mechatronics system design, and control systems.



OMAR S. ELFAYOUMI received the B.S. degree in mechanical engineering with a minor in mechatronics engineering from Arab Academy for Science and Technology and Maritime Transport (AASTMT), Alexandria, Egypt, in 2022. He is currently pursuing the M.S. degree with American University of Sharjah (AUS). His current research interests include mechatronics, robotics, computer vision, and machine learning.



BARA J. EMRAN (Member, IEEE) received the Ph.D. degree in mechanical engineering from The University of British Columbia, in 2019. He has worked with several leading robotics companies across a range of industries, contributing to the development and application of advanced robotic systems. In 2024, he joined the Department of Electrical Engineering, American University of Sharjah (AUS). His research interests lie at the intersection of advanced control systems and machine learning, with a particular focus on developing computationally efficient, and learning-based control algorithms.

...